ME 461 - Let Pico move it - part III

Recall:

Pinout of Pi Pico is also provided as a reference in Figure 1.

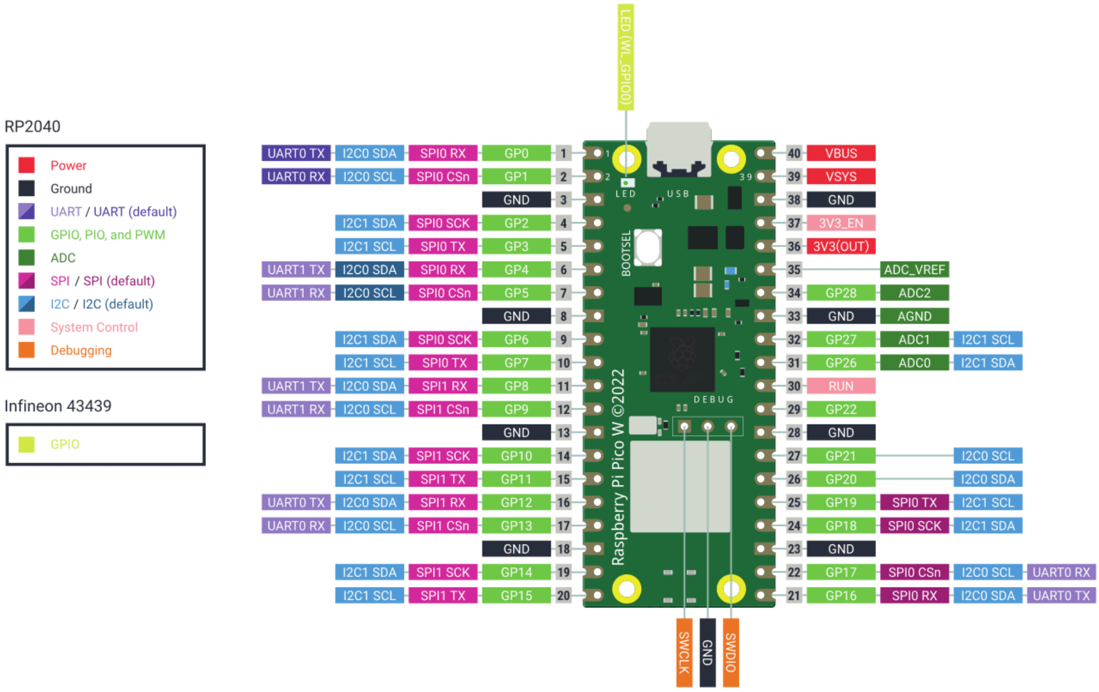


Figure 1: Raspberry Pi Pico W pinout

Rules of thumb: - would not hurt to remember 🙂

- Your code should
  - not crash no matter what
  - include sufficient comments
  - be as lean and short as possible
  - not have self-repeating parts
  - run smoothly and be responsive
- Try to use cut to length wires. Slayed network cables are a good single strand source.
- Everytime you take your breadboard etc out from your backpack or its container, manually check connections to make sure nothing is loose or disconnected. You will get faster in time :)
- Do not modify your circuit while it is powered on
- Do not rush into anything
- If something does not work as expected:
  - it is most probably not a bug but you are missing something.
  - Re-check the cabling and make sure that there are no loose connections
- If you feel a different odor coming from around your breadboard, power it down right away, and do not directly touch things to find out what happened.
- Based on definitions in Labs / Assignments, get used to taking incentives and making reasonable assumptions when certain details are not provided.

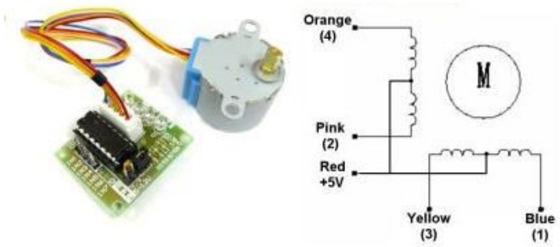
What 2 do?

STEP 1: Get ready

There we go, this time you are on your own.
Perform a literature survey to answer the following questions:

a) What is a unipolar stepper (In the rest of semester, a stepper motor can be referred to as *stepper*, or *SM*) ?
b) What is a bipolar stepper?
c) What is the difference between full- and half-stepping?
d) How can you increase the resolution of a stepper beyond half-stepping?
e) What is the slew rate?
f) What is the relation between torque and angular velocity (~ step rate)?

Note that 28BYJ 48 is meant in the context of this assignment when a step motor is mentioned. Figure 2 illustrates the 28BYJ 48 step motor and a switching sequence. Study this figure and at least try to answer some of the questions mentioned above in particular to the 28BYJ 48 step motor.
Also note that, similar to the RC motor lab, you are expected to use an external 5V supply here as well, which should be provided by the PCB you have if you have properly connected the external supply.



Half-Step Switching Sequence

Lead Wire Color	---> CW Direction (1-2 Phase)							
	1	2	3	4	5	6	7	8
4 Orange	-	-						-
3 Yellow		-	-	-				
2 Pink				-	-	-		
1 Blue						-	-	-

Figure 2. 28BYJ 48 step motor and ULN2003 driver

STEP 2: Mean business - right away

Playing with steppers is or can be fun! Especially when you do NOT use a library!!! So yes, you are **<u>not to use</u>** a library.

This way you will understand the importance of determining the sequence of coils.
The software tool you will write in this assignment will facilitate this process. A sample GUI similar to the one you should develop is illustrated in Figure 3. Well, 28BYJ 48 coil sequence is trivial, yet, using this GUI you will also have the chance to observe what happens if you misfire (i.e. excite coils in the wrong order).

This program should connect to Pi Pico as you like. Connection status problems should be handled by the program and feedback should be provided to the user; details are up to you.

Through this interface, users should be able to determine the actuation sequences for a given motor (Your first motor will be a set of 4 LEDs and related resistors, instead of an actual stepper motor :). If designed as shown and implemented as desired, this program should allow any combination of coil (or LEDs) actuation either step by step, or continuously at a given rate as shown in Figure 3.

The textbox on the left side of the GUI shown in Figure 3 contains desired actuation sequences. Only two sample actuation sequences are displayed. First sequence is “1 0 0 0”. This means that the 1st coil should be actuated.

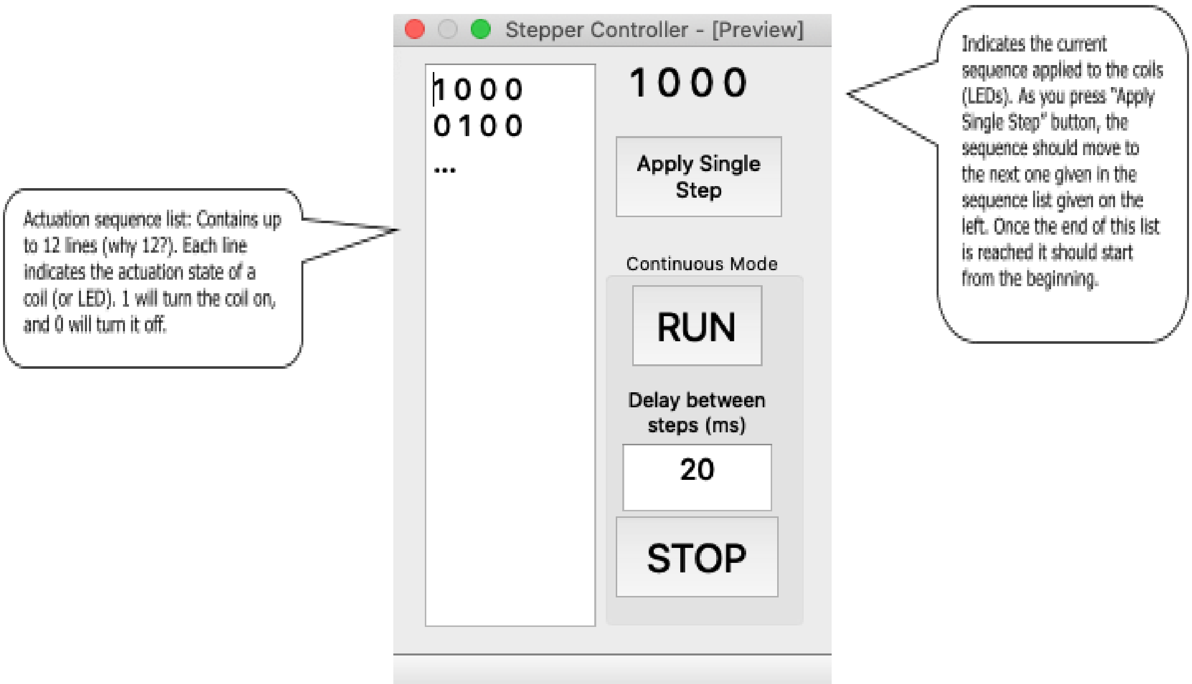


Figure 3. User interface for controlling stepper motors

If the user presses the “Apply Single Step” button, one sequence (or one line) from the text box will be read, parsed for separate coils, and sent down to Pi Pico for execution. If the user presses this button once more, the motor should move to the next sequence and so on. After executing the last line in the box, execution should roll back to the first line. The label on top of this button should show the currently executed sequence. Figure 3 illustrates the case where the button has been pressed for once.

After the user tries alternative sequences and finds a desired one composed of several lines, s/he should be able to run this at different rates. This will be done by the “Continuous Run” button. Once this button is pressed, a proper command should be sent to Pi Pico and it should start running the sequences given in the textbox continuously by pausing between consecutive sequences as indicated in the Delay textbox (A maximum delay value of 3000ms should be allowed). By changing this delay time, users will be able to set the pace of the stepper. Once Pi Pico is in continuous run mode, it should not depend on the PC.

When the user presses the “STOP” button, all coils should be deactivated. You can easily check this by trying to turn the motor shaft (or by inspecting that all LEDs are off).

STEP 3: No gear - No cry

Normally step motors do not come with a gearbox. 28BYJ 48 is preferred for all practical purposes. And yet, it has a built in gear train that has a gear ratio of 62:1. Or has it?
Write a simple command line tool (i.e. through REPL) that lets you **identify this ratio precisely.**

RULES of the GAME: enjoy your steps…

got fun?